

Main idea and results briefly

I experimented with several modeling approaches, including Ridge Regression, Lasso, ElasticNet, SVR with RBF kernel, neural-network-based models, and HuberRegressor. Among these methods, HuberRegressor performed the best overall, especially after applying robust preprocessing and feature engineering.

The final approach used:

Median imputation for missing values
RobustScaler to reduce the impact of outliers
Target clipping at the 1st and 99th percentiles
Feature engineering on x9, including square-root and cube-root transformations
K-fold validation to evaluate model stability

The best submitted model achieved a Public Score of 0.04674 and a Private Score of 0.03815, outperforming the baseline submission. The highest Private Score among all my submissions was achieved by the HGB model, with a Private Score of 0.11171.

Sofia ML Regression 2026 Spring				Late Submission	...
OverviewDataDiscussionLeaderboardRulesTeamSubmissions					
AllSuccessfulSelectedErrors				Recent	
Submission and Description		Private Score	Public Score	Selected	
	submission_mu_yingnan_20260608_try9_SVR_rbf_C100_epsilon50_gamma_scale_x9_sqrt_... Complete · 3d ago	0.00829	0.01681	☐	
	submission_mu_yingnan_20260608_try8_huber_clip_001_099_x9_sqrt_cbrt_stratified_kfold... Complete · 3d ago	0.03815	0.04674	☐	
	submission_huber_exp4C_Yingnan_Mu_best_epsilon_alpha.csv Complete · 3d ago	0.00425	0.00616	☐	
	submission_huber_Yingnan_Mu_exp4B_best_x9_transform.csv Complete · 3d ago	0.00370	0.00507	☐	
	submission_exp2_Yingnan_Mu_kfold_threshold_100.csv Complete · 3d ago	-0.44024	-1.20851	☐	
	hgb_Yingnan_Mu_submission_1.csv Complete · 4d ago	0.11171	-0.45514	☐	
	submission_mu_yingnan_20260531_try6_huber_clip_001_099_x9_sqrt_cbrt.csv Complete · 9d ago	0.03815	0.04674	☐	
	submission_mu_yingnan_20260531_try4.csv Complete · 11d ago	0.03815	0.04674	☐	
	submission_mu_yingnan_20260531_t1.csv Complete · 11d ago	0.03815	0.04674	☐	
	submission_mu_yingnan_20260531_try3.csv Complete · 11d ago	0.07785	0.04194	☐	
	submission_mu_yingnan_20260531_try2.csv Complete · 11d ago	0.07785	0.04194	☐	
	submission_mu_yingnan_20260531_try1.csv Complete · 11d ago	0.07785	0.04194	☐	
	submission_20260530_try1.csv Complete · 11d ago	0.06244	-0.09273	☐	

Part 1: Dataset Overview

```
# https://www.kaggle.com/competitions/sofia-ml-regression-2026-spring
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from datetime import datetime
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.model_selection import cross_val_score
```

```
train_df = pd.read_csv("/content/spring2026_kaggle_linear_regression_challenge_train.csv")
test_df = pd.read_csv("/content/spring2026_kaggle_linear_regression_challenge_test.csv")
sample_submission = pd.read_csv("/content/spring2026_sampleSubmission.csv")
```

```
train_df.head(5)
```

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	x10	x11
0	NaN	4.653737	-7.592557	1.714188	-0.622167	-2.758649	-3.576937	12.825621	-1.077701	-2.990621	-4.889358	7.697097
1	16.366673	-5.129070	-4.056004	-1.587978	7.676221	16.847336	-0.598568	-8.814040	-5.749997	4.542493	10.320050	7.853915
2	1.671239	10.997672	1.770459	1.324817	0.842158	4.314447	3.316170	-15.286950	-2.229412	-3.284181	-8.870790	-3.978508
3	-12.860650	-4.226342	1.288700	1.143941	8.863173	11.838897	-1.831860	-4.850510	5.983591	-1.335703	-17.712846	-16.387132
4	5.276754	-5.392047	1.255338	1.220763	-3.855019	6.816328	-3.477110	-3.808780	-2.662270	-8.349453	6.761591	-6.787475

Next steps: [Generate code with train_df](#) [New interactive sheet](#)

```
test_df.head(5)
```

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	x10	x11
0	2.328677	-13.124887	5.942170	1.835397	-24.313437	-19.432058	0.671548	-3.317686	-0.117173	-5.956024	12.210401	11.246734
1	5.084936	-1.242914	-3.351731	-0.155560	-21.027248	-21.594852	NaN	-10.462068	2.788057	-6.322131	-5.250770	18.786611
2	-1.569973	-11.989541	-2.684508	1.639759	-21.525546	6.482782	1.248830	-6.233198	-10.084921	0.503614	-7.351982	3.364457
3	-7.821665	2.262508	1.741962	2.652289	10.407001	5.325486	7.686760	-12.471255	-4.461678	-6.469431	-5.412491	-19.906638
4	9.980589	2.768668	-3.253775	2.158787	5.343083	22.917654	0.962534	-12.845845	-9.541650	11.526917	23.933020	-2.595816

Next steps: [Generate code with test_df](#) [New interactive sheet](#)

```
X_train = train_df.drop(columns=["Id", "target"])
y_train = train_df["target"]
```

```
X_test = test_df.drop(columns=["Id"])
```

Part 2: EDA

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Check feature shape
print("X_train shape:", X_train.shape)

# Compute correlation matrix
corr_matrix = X_train.corr()

# Create mask to hide upper triangle
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

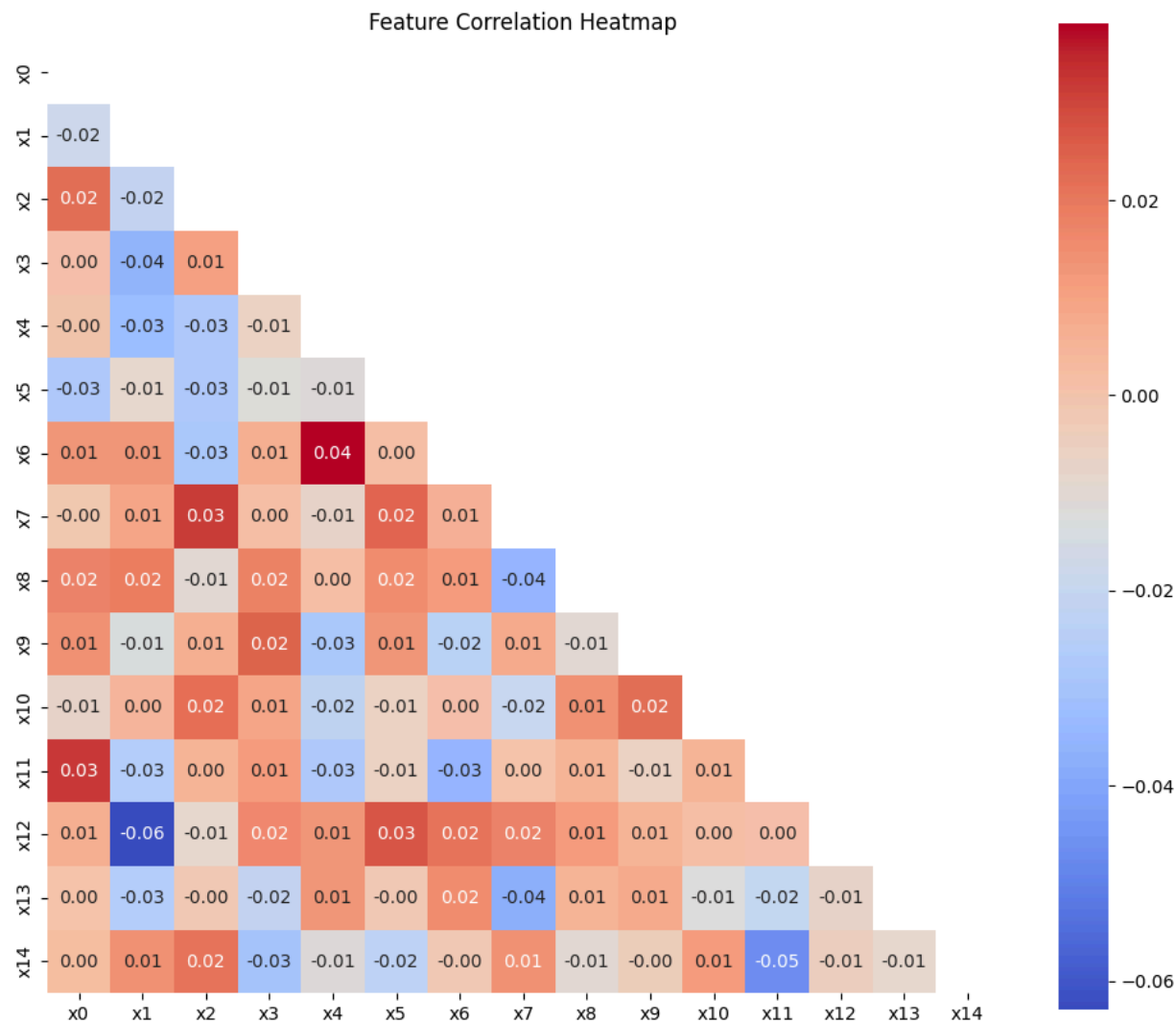
plt.figure(figsize=(12, 10))

sns.heatmap(
```

```
corr_matrix,
mask=mask,
annot=True,
cmap="coolwarm",
fmt=".2f",
square=True
)
```

```
plt.title("Feature Correlation Heatmap")
plt.show()
```

X_train shape: (2500, 15)



The correlation heatmap shows that most features have very weak pairwise correlations, with values close to 0. This means there is no strong multicollinearity among the input features. However, it also suggests that the dataset may not contain obvious linear feature relationships, which motivated me to test both linear and non-linear regression models.

```
train_corr = X_train.copy()
train_corr["target"] = y_train

target_corr = train_corr.corr()["target"].drop("target").sort_values(
    key=abs,
    ascending=False
)

print(target_corr)
```

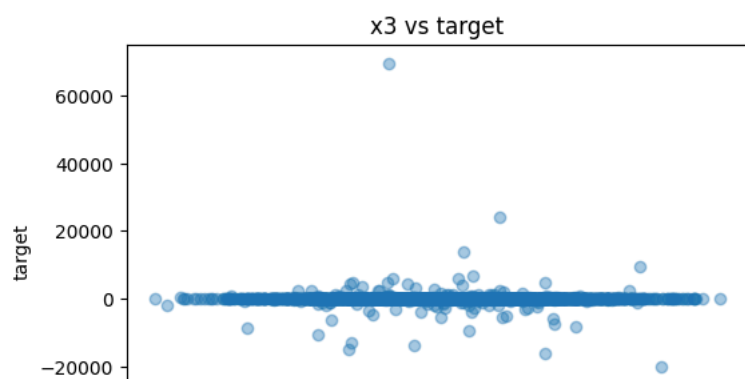
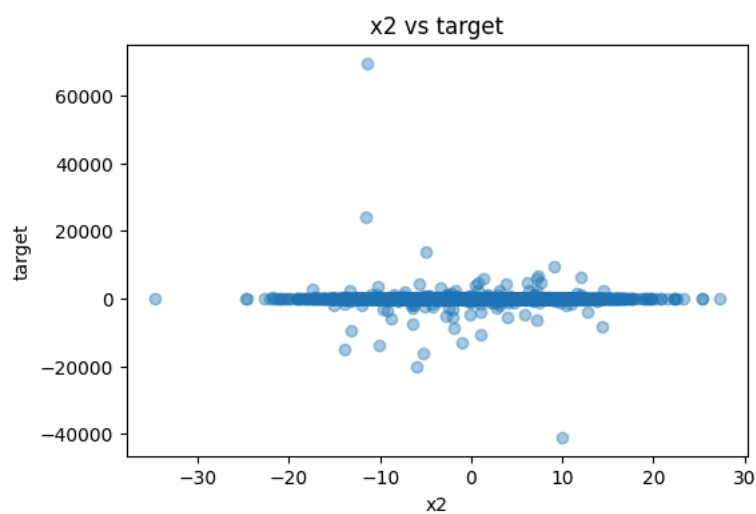
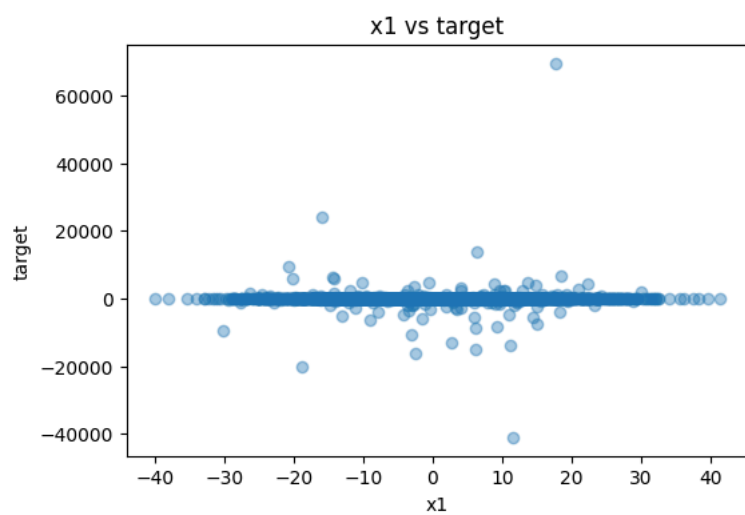
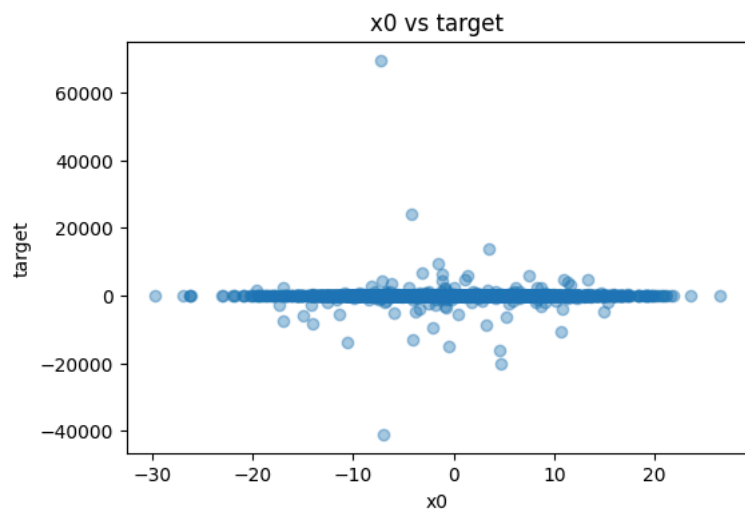
```
x9    0.232323
x8   -0.047537
x4    0.040760
```

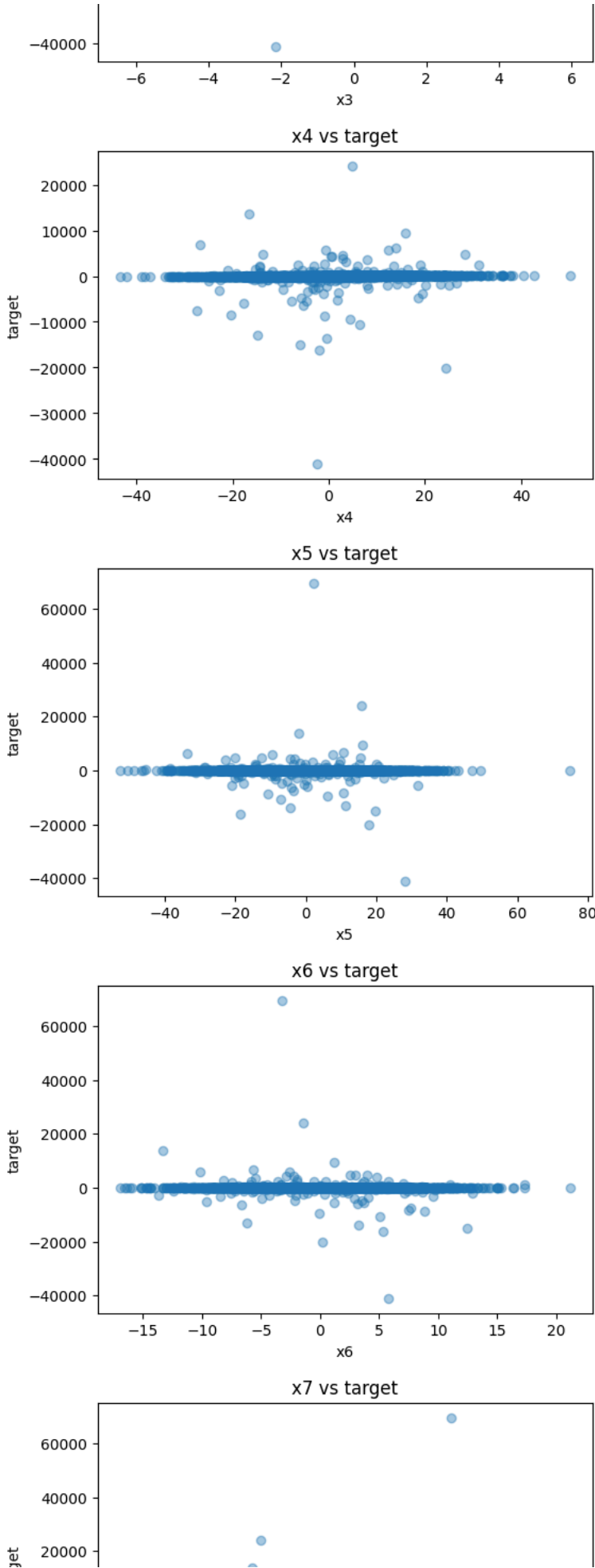
```
x6      -0.040455
x11     0.027480
x5       -0.026242
x10     -0.023372
x2       -0.022743
x7       0.020401
x12     0.014869
x1       0.004935
x0       0.003338
x14     -0.003132
x13     0.002873
x3       -0.001986
Name: target, dtype: float64
```

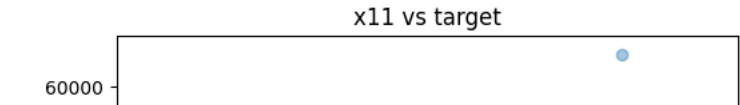
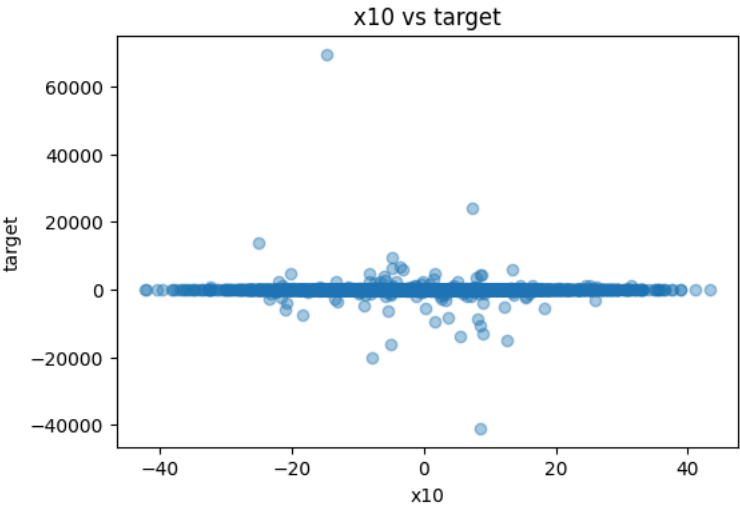
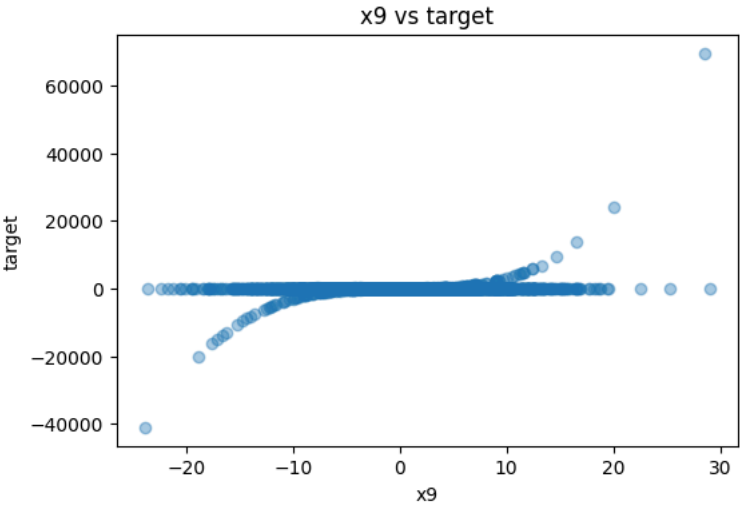
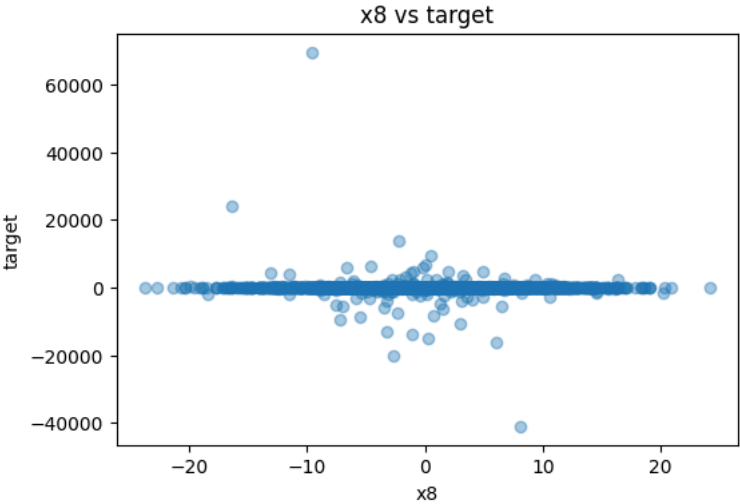
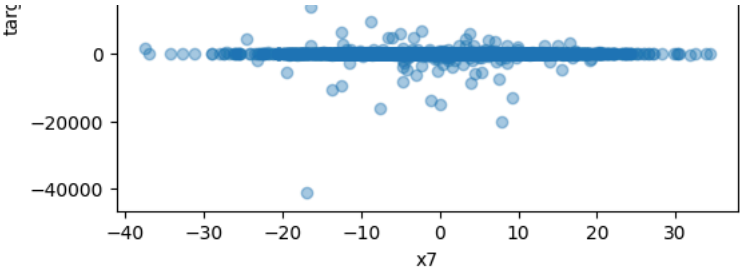
Among all features, x9 has the highest correlation with the target, with a correlation value of 0.2323. All other features have much weaker correlations, mostly close to 0. This suggests that x9 may be the most informative feature in the dataset, which motivated me to perform additional feature engineering on x9, such as square-root and cube-root transformations.

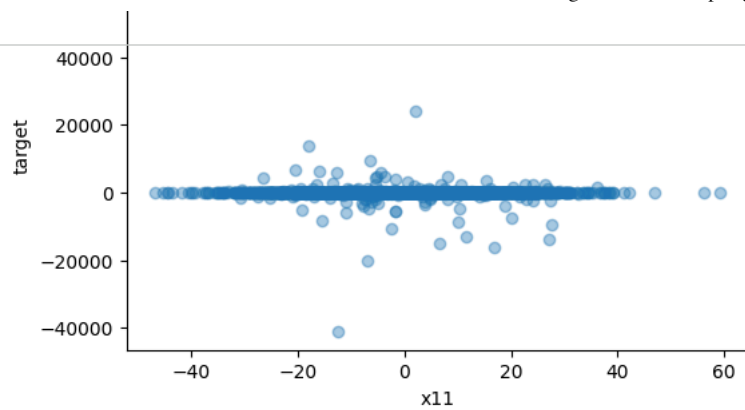
```
import matplotlib.pyplot as plt

for col in X_train.columns:
    plt.figure(figsize=(6, 4))
    plt.scatter(X_train[col], y_train, alpha=0.4)
    plt.xlabel(col)
    plt.ylabel("target")
    plt.title(f"{col} vs target")
    plt.show()
```

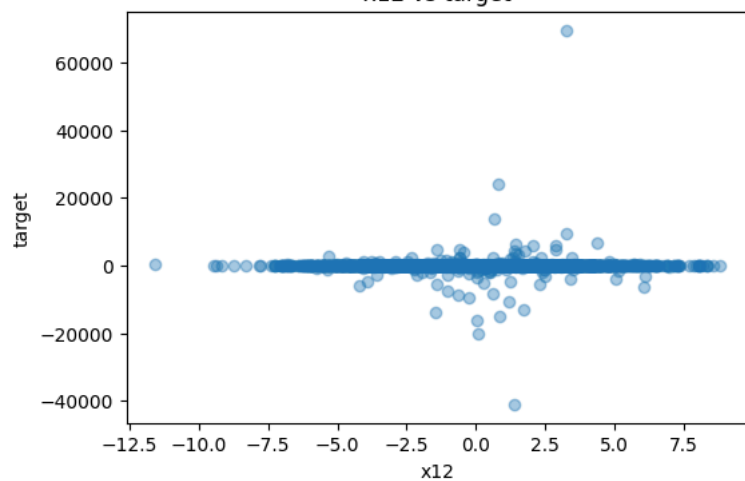





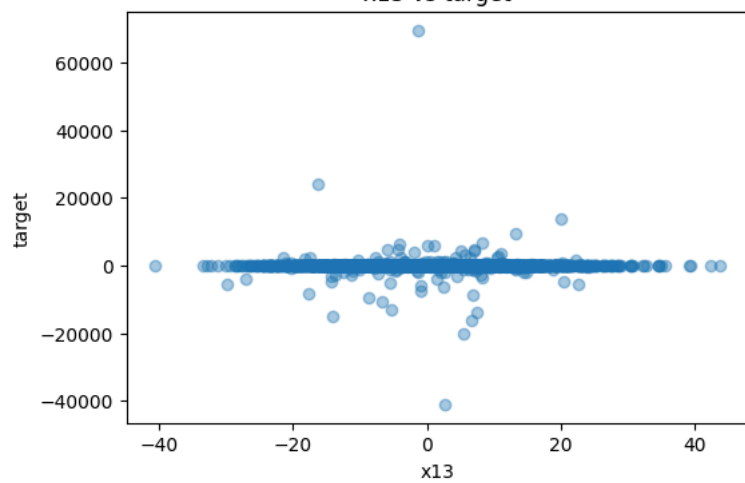




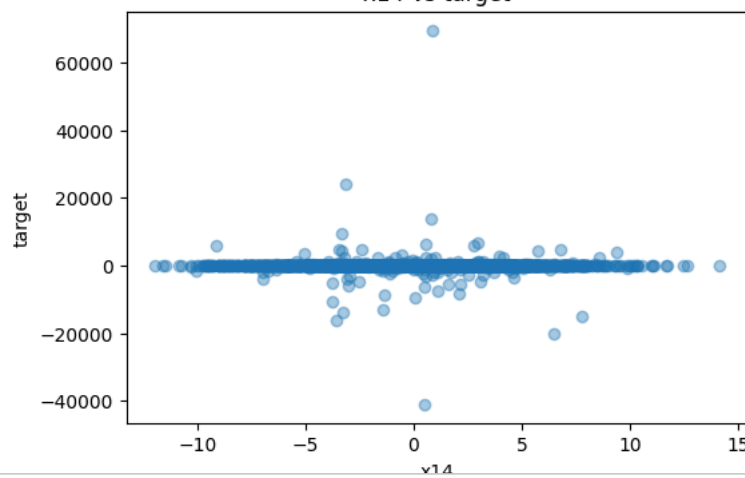
x12 vs target



x13 vs target



x14 vs target



```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.scatter(range(len(y_train)), y_train, alpha=0.6)

plt.title("Target Distribution")
plt.xlabel("Sample Index")
plt.ylabel("Target")
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt

# 1. Basic statistics
print("Target basic statistics:")
print(y_train.describe())
print()
print("Skewness:", y_train.skew())
print("Kurtosis:", y_train.kurtosis())

# 2. Histogram
plt.figure(figsize=(8, 5))
plt.hist(y_train, bins=100)
plt.xlabel("target")
plt.ylabel("Count")
plt.title("Target Distribution")
plt.show()

# 3. Boxplot
plt.figure(figsize=(8, 4))
plt.boxplot(y_train, vert=False)
plt.xlabel("target")
plt.title("Target Boxplot")
plt.show()

# 4. arcsinh transformed target
y_train_asinh = np.arcsinh(y_train)

plt.figure(figsize=(8, 5))
plt.hist(y_train_asinh, bins=100)
plt.xlabel("arcsinh(target)")
plt.ylabel("Count")
plt.title("Transformed Target Distribution")
plt.show()

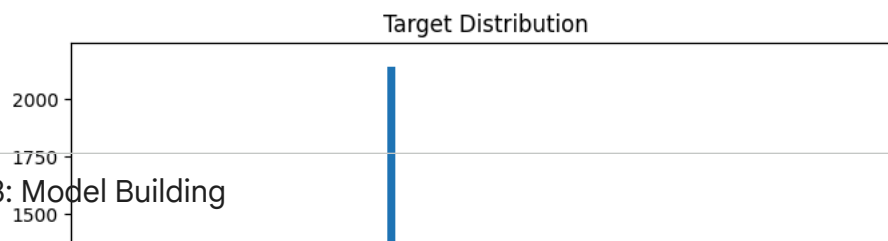
print()
print("Arcsinh target statistics:")
print(y_train_asinh.describe())
print()
```

```
print("Skewness after arcsinh:", y_train_asinh.skew())  
print("Kurtosis after arcsinh:", y_train_asinh.kurtosis())
```



```
Target basic statistics:
count      2500.000000
mean       -22.043467
std        1978.177302
min        -41008.015927
25%        -43.166743
50%        -0.769750
75%        42.000951
max        69628.199874
Name: target, dtype: float64
```

```
Skewness: 13.587995421968362
Kurtosis: 707.8060354831437
```



Part 3: Model Building

```
# Step: Model building - M1: LinearRegression

# 1. Using the median from the X_train dataset to build the model - R2:
model = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("linear_regression", LinearRegression())
])

scores = cross_val_score(model, X_train, y_train, cv=5, scoring="r2")

print("CV R2 scores:", scores)
print("Mean CV R2:", scores.mean())
print("Std CV R2:", scores.std())
```

```
CV R2 scores: [ 0.05512204 -0.01115937 -0.12938069  0.03584363 -0.07543408]
Mean CV R2: -0.025001693361575428
Std CV R2: 0.0689231102688884
```

target

Target Boxplot

I first built a simple Linear Regression model as the baseline experiment. Missing values were handled using median imputation, and the model was evaluated with 5-fold cross-validation using R^2 as the metric.

```
# Step: Model building - M2: Ridge + x9 polynomial + RobustScaler

import numpy as np
import pandas as pd

from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import RobustScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score

# =====
# Model 2: Ridge + x9 polynomial + RobustScaler
# =====

# 1. Copy X_train only
X_train_1 = X_train.copy()

# 2. Feature engineering: add polynomial features for x9
X_train_1["x9_squared"] = X_train_1["x9"] ** 2
X_train_1["x9_cubed"] = X_train_1["x9"] ** 3

print("X_train_1 shape:", X_train_1.shape)

# 3. Build pipeline
# Every CV fold will:
# - fit median imputer on training fold
# - transform validation fold using that imputer
# - fit RobustScaler on training fold
# - transform validation fold using that scaler
# - train Ridge model
```

```

model_1 = make_pipeline(
    SimpleImputer(strategy="median"),
    RobustScaler(),
    Ridge(alpha=1.0)
)

# 4. Cross-validation
scores_1 = cross_val_score(
    model_1,
    X_train_1,
    y_train,
    cv=5,
    scoring="r2"
)

# 5. Print results
print("Model 1: Ridge + x9 polynomial + RobustScaler")
print("CV R2 scores:", scores_1)
print("Mean CV R2:", scores_1.mean())
print("Std CV R2:", scores_1.std())

X_train_1 shape: (2500, 16)
Model 1: Ridge + x9 polynomial + RobustScaler
CV R2 scores: [ 0.06852993 -0.08342962 -0.17604783  0.03311524 -0.12573111]
Mean CV R2: -0.05671267823080528
Std CV R2: 0.0932448212401032

```

After the simple Linear Regression baseline, I tested Ridge Regression with additional feature engineering on x9. Since EDA showed that x9 had the strongest correlation with the target, I added polynomial transformations of x9 and used RobustScaler to reduce the impact of outliers. Missing values were still handled before modeling.

```

import numpy as np
import pandas as pd

from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import PowerTransformer, StandardScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score

# =====
# Model 3: Ridge + x9_squared + PowerTransformer
# =====

# 1. Copy X_train only
X_train_3 = X_train.copy()

# 2. Feature engineering: add x9 polynomial feature
# Based on previous result, we only add x9_squared, not x9_cubed
X_train_3["x9_squared"] = X_train_3["x9"] ** 2

print("X_train_3 shape:", X_train_3.shape)
print(X_train_3[["x9", "x9_squared"]].describe())

# 3. Build pipeline
# Every CV fold will:
# - fill NaN using median
# - transform features using Yeo-Johnson PowerTransformer
# - scale transformed features
# - train Ridge model
model_3 = make_pipeline(
    SimpleImputer(strategy="median"),
    PowerTransformer(method="yeo-johnson"),
    StandardScaler(),
    Ridge(alpha=1.0)
)

# 4. Cross-validation
scores_3 = cross_val_score(
    model_3,
    X_train_3,
    y_train,

```

```

cv=5,
scoring="r2"
)

# 5. Print results
print("Model 3: Ridge + x9_squared + PowerTransformer")
print("CV R2 scores:", scores_3)
print("Mean CV R2:", scores_3.mean())
print("Std CV R2:", scores_3.std())

X_train_3 shape: (2500, 16)
      x9  x9_squared
count 2470.000000 2470.000000
mean  -0.217409  49.069504
std    7.003006  71.565100
min   -23.885863  0.000021
25%   -4.917531  5.376485
50%   -0.250908  22.864566
75%    4.560281  65.273616
max    28.984316  840.090553
Model 3: Ridge + x9_squared + PowerTransformer
CV R2 scores: [ 0.05522214 -0.01871673 -0.12876721  0.03543397 -0.07708703]
Mean CV R2: -0.026782970538279095
Std CV R2: 0.06869080606314086

```

- For the third model, I continued using Ridge Regression but simplified the x9 feature engineering by only adding x9_squared. The motivation was to capture a possible non-linear relationship between x9 and the target without adding too many polynomial features.

The modeling pipeline included median imputation, Yeo-Johnson PowerTransformer, StandardScaler, and Ridge Regression. The PowerTransformer was used to make the feature distributions more stable and closer to normal before fitting the linear model.

```

import numpy as np
import pandas as pd

from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import RobustScaler
from sklearn.linear_model import Ridge
from sklearn.compose import TransformedTargetRegressor
from sklearn.model_selection import cross_val_score

# =====
# Model 4: Ridge + x9_squared + RobustScaler + arcsinh target
# =====

# 1. Copy X_train only
X_train_4 = X_train.copy()

# 2. Feature engineering: add x9_squared only
# Based on previous result, we do NOT add x9_cubed
X_train_4["x9_squared"] = X_train_4["x9"] ** 2

print("X_train_4 shape:", X_train_4.shape)
print(X_train_4[["x9", "x9_squared"]].describe())

# 3. Build base model for X
base_model_4 = make_pipeline(
    SimpleImputer(strategy="median"),
    RobustScaler(),
    Ridge(alpha=1.0)
)

# 4. Wrap the base model with target transformation
model_4 = TransformedTargetRegressor(
    regressor=base_model_4,
    func=np.arcsinh,
    inverse_func=np.sinh
)

# 5. Cross-validation
scores_4 = cross_val_score(
    model_4,
    X_train_4,

```